Lab 2.a derive walkthrough — “Your turn: derive a rule”, step by step

The derivation block (“Your turn — derive a rule” through “Checkpoint 2”), written down. Use it to complete the block at your own pace if you missed the live session, or — if you’re presenting — as the helper that makes sure no beat gets skipped (see the last section).

> **If the live session is still ahead of you, stop here.** This document replays a discovery the exercise exists to let you make yourself. Reading it before the session costs you the exercise — the same reason the deck isn’t handed out in advance. Do the session, then use this to catch up on anything that didn’t land.

This is the one block in Lab 2 with no answer key. The walkthrough gives you the method, one fully worked example, and hints shaped as questions — it deliberately does **not** hand you a finished clause list, because a rule you derived gets followed and a rule you copied gets routed around. Everything here is individual work in your own copy; if the class ran live, there is one shared calibration step (step 5) you can still do asynchronously.

Before you start: your own copy on branch `feat/governance`, the ADR from the previous section staged (Checkpoint 1 passed), and ~40 minutes. Have something to write clauses in — a scratch file is fine; they land in a real document in step 6.

Step 1 — Read the three fixes (≈5 min)

Three fixes for the same bug — Bug #4 from Lab 1.b (`stats.service.ts`: `COMPLETED_STATUS` was `'completed'`, the domain status is `'done'`, so “Completed this week” always showed 0). All three were authored for this exercise. All reviewed. All merged. **All CI-green.**

Fix A — fixed under pressure and shipped. “Test to follow” — it never did:

```
- const COMPLETED_STATUS: string = 'completed';
+ const COMPLETED_STATUS: string = 'done';
```

Fix B — fixed, then a test written afterwards. It passes — and it also passes on the UNfixed code:

```
- const COMPLETED_STATUS: string = 'completed';
+ const COMPLETED_STATUS: string = 'done';
```

```
// stats.service.test.ts (new)
it('counts completed todos this week', () => {
  const summary = statsService.summary();
  expect(summary.completedThisWeek).toBe(0); // fixture's done todo is dated Jan 2026
});
```

(Read that assertion twice. The bug returned 0, and the fixture's only done todo is months old — so the test asserts 0 either way. It is a real test, it runs, and it proves nothing.)

Fix C — a failing test written first; the fix made it pass:

```
it('counts todos completed in the last 7 days', () => {
  todosService.complete('todo_a'); // stamps completedAt = now
  expect(statsService.summary().completedThisWeek).toBe(1); // RED before the fix
});
```

…then the same one-line fix, and the test goes green. (This is what the cohort's own 1.b PRs did.)

Now answer two questions per fix, honestly, before moving on — write the verdicts down:

1. **Would you merge it?** (The live session ran this as a poll. B gets merged more often than people expect — CI is green and there's a test file in the diff.)
2. **What, exactly, convinces you the bug is actually fixed?**

Hold your answers — you're about to spend them.

Step 2 — Verdict the four seed clauses (≈4 min)

These are the four seed clauses the class starts from. For each one: verdict it **keep / tighten / drop**, and tag it **MECHANICAL** (a script, given only the diff, could return pass/fail) or **FUZZY** (it needs a person — if you need to know *intent*, it's fuzzy).

> **The catch-test — the whole method in one rule:** a clause earns **keep** only if you can name **which of A / B / C it catches**. *Tighten* means more checkable, not more strict.

1. A fix ships with a test
2. The test failed before the fix
3. Test lives next to its module
4. Full suite green before the PR

Seed 1, fully worked, as the model:

> *Verdict:* ***tighten***. "A fix" is not observable — which changes count? Tightened: **"a change under `server/src/**` ships with a test in the same PR."** Now a script could check it from the diff alone (did the diff touch `server/src`? did it touch a `*.test.ts`?), so the tag is

> **MECHANICAL**. Catch-test: it catches **A** (no test in the diff). Notice — say it out loud — what it does *not* catch.

Seeds 2–4, hints only — write your verdict before reading the next hint:

- **Seed 2** — run each fix through it. Which one does it catch that seed 1 missed? Then try to tag it: what would a script *read from a merged diff* to decide whether the test failed before the fix existed? Take that question seriously; don't wave it off. This seed is the most important line on the page.
- **Seed 3** — run the catch-test honestly. If it catches none of A/B/C, your options are **drop**, or **keep for a reason the catch-test doesn't cover** (repo hygiene is a real reason) — but then say so explicitly. Both are defensible; pretending it catches something is not.
- **Seed 4** — all three fixes were CI-green, so what does it catch? A clause can survive as a *floor* (it catches a fourth fix nobody drew). Tag it — this one's easy to tag.

Step 3 — Add your own clauses (≈3 min)

What did the seeds miss? One or two more, tagged. More than 6 clauses total means they're too small. Prompts that produce good ones:

- Is there a **kind of change** your tightened seed 1 shouldn't apply to? (An exemption is a clause — and a good one here becomes a CI path filter in 2.b.)
- Look at your step-1 answer to "what convinces you the bug is fixed?" — is that conviction written down anywhere in your list yet?
- What would catch a fix that's working on someone's machine but **never becomes a PR at all**? (Several real 1.b fixes did exactly this.)

> **Self-check before moving on — run the catch-test on your whole list:** name the clause that catches A. Now name the clause that catches **B**. If nothing on your list catches B, you are not done: look again at what separates B from C — the difference between them *is* a clause, and it's the one this exercise exists to make you find.

Step 4 — Mark your least-sure clause (≈2 min)

Pick the one clause you're **least sure** about — wrong wording, uncheckable, might be a bad idea — and mark it. Doubt is the interesting output: at the class compare, the least-sure clauses are the ones read out loud, because where lists disagree is where the standard is actually being made. If you're working solo, your least-sure clause is the first thing to raise in Slack.

Step 5 — The compare: calibrate against AGREED (≈4 min)

In the live session, everyone pastes their list into the shared Doc under their own name, the lists go side by side, and the class agrees **one AGREED list** — calibration, not committee work. The tiebreak when lists conflict: *the stricter clause wins, unless someone can show it false-positives on real code in this repo.*

Working asynchronously:

- **The session already happened:** get the shared Doc link from Slack and read the **AGREED** section. Compare it to your list clause by clause. AGREED is the standard you'll write up in step 6; where your list disagrees, your version is recorded — not erased — as a **dropped clause with a reason** (see step 6). That's how disagreement with a live standard is recorded on a real team.
- **No session, fully self-guided:** your list *is* the standard — the step-3 self-check is your calibration. Post your list in Slack so a reviewer sees it before step 6.

One more calibration question, answered before you move on: **which of the five artifacts is this list?** It feels like a decision — but run the quick test: it must *always hold*, and you can say *how it's checked*. That's an **NFR** — present-tense law, revised in place. (If adopting it ever becomes genuinely contested, *that* argument would earn an ADR; and "how do I write a failing test first?" is recipe material.)

Step 6 — Write it up as an NFR (≈20 min)

Turn the AGREED clauses (or your own list, if self-guided) into `docs/nfr/0002-test-discipline.md` in your copy — your wording, your document. Copy the headings and tone of NFR-0001, and add the per-clause check list:

```
## The rules

1. A change under `server/src/**` ships with a test in the same PR.
   **Checked by:** CI — the diff must touch a `*.test.ts` when it touches
        `server/src`.
2. The test must have failed before the fix.
   **Checked by:** — NOT MECHANICALLY CHECKABLE — decision: human gate (reviewer
        asks) — <name>
```

The rules of the document:

- **Every clause states how compliance is checked.** A clause that can't be checked is marked `NOT MECHANICALLY CHECKABLE`, carries a decision (**accepted risk · human gate · dropped**), and a **name**. An unmarked unenforceable clause is the dangerous kind.
- **Dissent goes in as a dropped clause.** If your list disagreed with AGREED, write the AGREED version in the rules and record yours under a `## Dropped clauses` heading with the reason it lost. That sentence is worth more than the clause.
- Resist "Checked by: code review" as a reflex. For each clause, ask once: could a script check *this* from the diff? Review is the human gate — a decision, not a default.

Then wire it into `CLAUDE.md` exactly as the handout's step 4 describes (trigger-table row + binding line), commit, and open your PR.

Step 7 — Checkpoint (the same bar as the live session)

Passed means: your NFR names how **each** clause is checked — and which ones can't be. The live check was "paste one clause + its Checked-by line into chat; fuzzy ones include the decision and a name." Async: your PR is the paste — and every acceptance criterion still applies, including the 3–5 sentence reflection note in the PR and peer review (ask in Slack for a reviewer).

Fast finisher? The stretch is the store boundary (`db/store.ts` imports): same method — derive, tag, write it up as `docs/nfr/0003-db-access-boundary.md` .

Common wobbles

Wobble	The move
Every seed feels like "keep"	You're grading the wording, not the rule. Run the catch-test: a clause that catches nothing has to argue for its life
Nothing on my list catches B	Reread B's assertion and ask what C did that B didn't — *before* the fix existed. The difference is your missing clause
My clause catches B but I can't tag it	Congratulations — you found the finding. Tag it FUZZY, mark the decision and a name in step 6, and don't resolve it today: 2.b opens on exactly this
Polished wording, no verdicts after 5 minutes	Editing beats authoring. Verdict all four seeds in plain words first; wording is step-6 work
My list disagrees with AGREED	Good. Write the AGREED version, record yours as a dropped clause with a reason — recorded, not erased
Every clause ends "Checked by: code review"	The human gate as a reflex, not a decision. Per clause, once: could a script check this from the diff?
"Just tell me the right list"	There isn't one until you write it. That's the difference between this and every rule you've been handed

---

If you're presenting — the don't-miss list

The beats that make the live block work, in order. (The full run sheet with timings and recovery moves is in the instructor material; this list is the "did I skip one?" check.)

- [ ] Shared Doc prepared **before the call**: the three diffs at the top, the four seeds once, one heading per student **by name**, an **AGREED** heading at the bottom; sharing set to "anyone with the link can edit"; link pasted in chat at the divider
- [ ] Call norms said once at the divider — mics muted, quiet stretches are deliberate, everything you make today is yours; the compare is calibration, not committee work
- [ ] Four minutes of **silent** reading announced as silent — then the merge **poll** (A? B? C?), results shown with no commentary

- [ ] The clock presented in **one minute**: verdict seeds → add your own → mark least-sure → paste under your name
- [ ] Say once before the clock starts: “stuck? Slack me — Meet chat has no DMs”
- [ ] Whole-call chat nudges at **~3 / ~6 / ~8 min** — never aimed at a name; strugglers get a quiet Slack ping instead
- [ ] Fast finishers get the store-boundary stretch **via Slack**, never announced mid-derive
- [ ] Converge with the **Doc on screen**, not the deck: fast wins first (clauses most lists share), then two or three **least-sure** clauses read on mic; type AGREED where everyone watches
- [ ] The tiebreak stated as a rule, once: stricter wins unless it false-positives on real code in this repo
- [ ] The **artifact beat** as a chat waterfall (“which of the five artifacts is this list? type, hold, 3-2-1, send”) — do not skip it
- [ ] The dissent line when someone’s clause loses: AGREED is what you write up; yours goes in as a dropped clause with a reason
- [ ] The mature-NFR compare slide only **after** AGREED is committed — earlier, it’s an answer key
- [ ] Before ending the call: **copy the chat and the AGREED list out of Meet** — chat is deleted when the call ends